

Manual de Programador — Vertice Muisca (VerticeMusicasWeb)

Campo	Valor
Audiencia	Desarrolladores que mantienen, modifican o extienden el sistema
Repositorio	Ingenieria softw/
Proyecto ejecutable	VerticeMusicasweb/
Stack	.NET 8, ASP.NET Core MVC, EF Core, SQLite, Razor, Bootstrap 5
Arquitectura	Monolito en capas (no microservicios)
Última actualización documentada	Entrada de inventario + informes (PDF/impresión por sección, QuestPDF)

Tabla de contenido

- [1. Introducción para desarrolladores](#)
 - [2. Arquitectura interna del sistema](#)
 - [3. Organización del proyecto y estructura de carpetas](#)
 - [4. Explicación de módulos y componentes](#)
 - [5. Explicación de clases, funciones y servicios principales](#)
 - [6. Flujo interno de la aplicación](#)
 - [7. Explicación de rutas y endpoints](#)
 - [8. Manejo de autenticación y autorización](#)
 - [9. Configuración del entorno de desarrollo](#)
 - [10. Dependencias del proyecto](#)
 - [11. Convenciones de código utilizadas](#)
 - [12. Patrones de diseño identificados](#)
 - [13. Conexión con base de datos](#)
 - [14. Manejo de errores y logs](#)
 - [15. Cómo agregar nuevas funcionalidades](#)
 - [16. Cómo crear nuevos endpoints o módulos](#)
 - [17. Recomendaciones para mantenimiento](#)
 - [18. Buenas prácticas del proyecto](#)
 - [19. Problemas comunes y soluciones](#)
 - [20. Conclusiones](#)
-

1. Introducción para desarrolladores

Este manual describe **cómo está construido el código** de Vertice Muisca para que un programador pueda:

- Localizar rápidamente dónde cambiar una regla de negocio.
- Añadir módulos siguiendo los patrones existentes.
- Evitar regresiones en stock, ventas e informes.
- Diagnosticar fallos de compilación, migraciones y vistas.

1.1 Qué NO es este proyecto

Expectativa incorrecta	Realidad en el código
Microservicios con API Gateway	Un solo proceso ASP.NET Core
API REST documentada con Swagger	MVC + algunas respuestas <code>Json()</code>
Autenticación JWT / Identity	Sin login ni roles
SQL Server / PostgreSQL configurado	SQLite archivo local

1.2 Punto de entrada del desarrollador

Siempre comenzar por:

1. `VerticeMusicaWeb/Program.cs` — DI, BD, pipeline, migraciones.
2. `VerticeMusicaWeb/Data/AppDbContext.cs` — modelo relacional.
3. El **controlador** del módulo a tocar.
4. El **servicio** asociado (si existe).
5. Las **vistas** en `Views/{Modulo}/`.

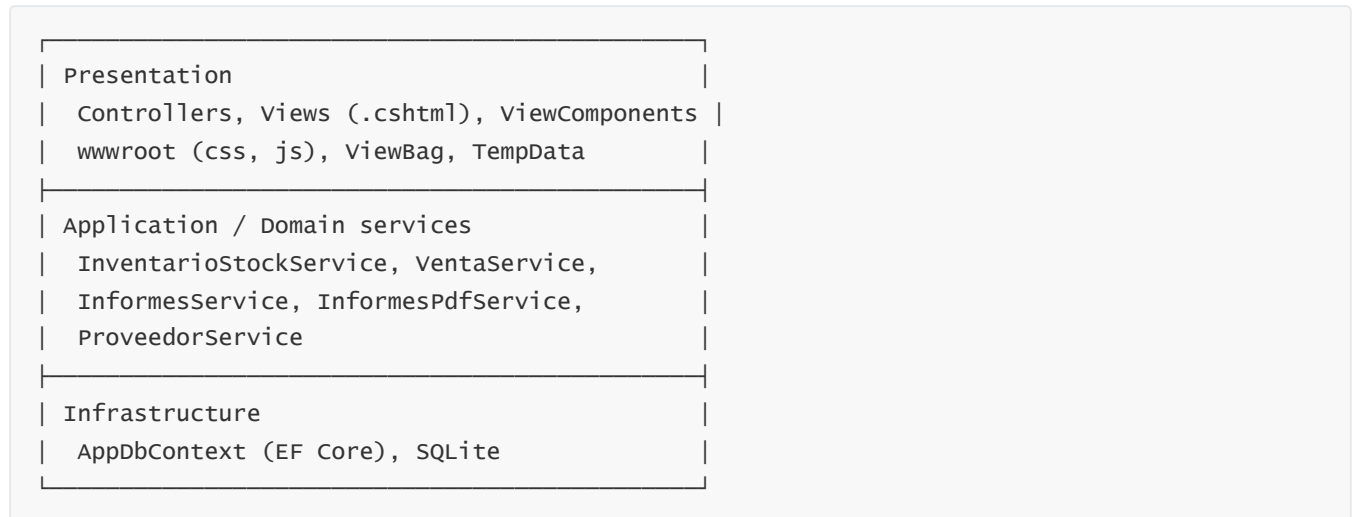
1.3 Nomenclatura inconsistente (conocida)

Ubicación	Nombre
<code>.csproj</code> → <code>AssemblyName</code> , <code>RootNamespace</code>	<code>VerticeMuiscaWeb</code>
Código C# (namespace)	<code>VerticeMusicaWeb</code>
Migraciones EF	<code>VerticeMuiscaWeb.Migrations</code>
Proceso Windows al ejecutar	<code>VerticeMuiscaWeb.exe</code>

Al buscar ensamblados o procesos, probar ambas variantes ortográficas.

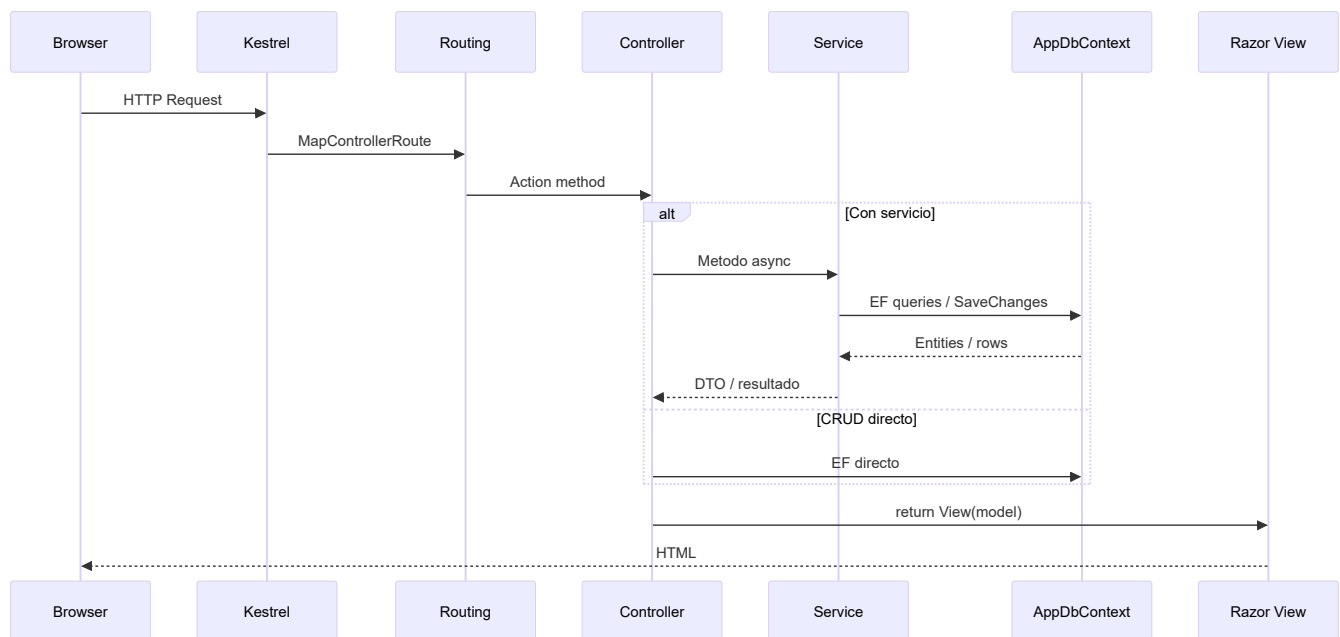
2. Arquitectura interna del sistema

2.1 Capas



No hay proyectos separados por capa (todo en un único `.csproj`).

2.2 Ciclo de vida de una petición MVC



2.3 Registro de dependencias (Program.cs)

```
var mvcBuilder = builder.Services.AddControllerswithViews();
if (builder.Environment.IsDevelopment())
{
    mvcBuilder.AddRazorRuntimeCompilation();
}

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite($"Data Source={dbPath}"));
```

```

builder.Services.AddScoped<InventarioStockService>();
builder.Services.AddScoped<VentaService>();
builder.Services.AddScoped<InformesService>();
builder.Services.AddScoped<InformesPdfService>();
builder.Services.AddScoped<ProveedorService>();

```

Servicio	Lifetime	Dependencias
AppDbContext	Scoped (por defecto EF)	—
InventarioStockService	Scoped	AppDbContext
VentaService	Scoped	AppDbContext, InventarioStockService
InformesService	Scoped	AppDbContext
InformesPdfService	Scoped	Sin BD; recibe InformesViewModel ya cargado
ProveedorService	Scoped	AppDbContext

3. Organización del proyecto y estructura de carpetas

```

Ingenieria softw/
├─ Ingenieria softw.sln
├─ BasesDeDatos/
│   └─ DBinventario_s1y2.mwb          # Modelo ER de referencia (no usado en runtime)
├─ verticeMusicasweb/
│   ├── Program.cs                    # ★ Arranque
│   ├── verticeMusicasweb.csproj
│   ├── appsettings.json
│   ├── appsettings.Development.json
│   ├── vertice_musicas.db            # ★ BD SQLite en desarrollo
│   ├── Controllers/                  # ★ Acciones HTTP
│   ├── Data/
│   │   └─ AppDbContext.cs            # ★ Mapeo EF
│   ├── Models/                       # Entidades + ViewModels + DTOs fila
│   ├── Services/                     # ★ Lógica de negocio
│   ├── ViewComponents/
│   └── views/
│       ├── Shared/
│       │   ├── _Layout.cshtml        # Menú + TempData + AlertaStock
│       │   └─ Components/AlertaStock/
│       ├── Dashboard/
│       ├── Categorias/
│       ├── Productos/
│       ├── Proveedores/
│       ├── Inventario/
│       ├── ConsultaStock/
│       ├── ventas/
│       └─ Informes/

```

```

├─ Migrations/                # EF migrations
├─ wwwroot/
│   └─ css/                   # site.css, dashboard.css, informes.css,
proveedores.css
│   └─ js/                    # proveedores.js, site.js
├─ Properties/launchSettings.json
├─ Iniciar-Vertice-Web.ps1
└─ build-safe.ps1

```

3.1 Convención de vistas Razor

Ruta de vista	Controlador	Acción típica
<code>Views/{NombreControladorSinSufijo}/{Accion}.cshtml</code>	<code>ProductosController</code>	<code>Views/Productos/Index.cshtml</code>

`_viewStart.cshtml` aplica `Layout = "_Layout"` a todas las vistas.

`_viewImports.cshtml`:

```

@using VerticeMusicasWeb
@using VerticeMusicasWeb.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

```

4. Explicación de módulos y componentes

4.1 Mapa de módulos funcionales

Módulo	Controlador	Servicio	Acceso a datos
Inicio	<code>DashboardController</code>	—	<code>AppDbContext</code> directo
Categorías	<code>CategoriasController</code>	—	Directo
Productos	<code>ProductosController</code>	—	Directo
Proveedores	<code>ProveedoresController</code>	<code>ProveedorService</code>	Vía servicio
Inventario	<code>InventarioController</code>	<code>InventarioStockService</code>	Vía servicio
Consulta stock	<code>ConsultaStockController</code>	<code>InventarioStockService</code>	Vía servicio
Ventas	<code>ventasController</code>	<code>VentaService</code>	Vía servicio
Informes	<code>InformesController</code>	<code>InformesService</code>	Vía servicio

4.2 ViewComponent global

Archivo: `ViewComponents/AlertaStockViewComponent.cs`

Vista: `Views/Shared/Components/AlertaStock/Default.cshtml`

Invocación: `_Layout.cshtml` → `@await Component.InvokeAsync("AlertaStock")`

Muestra barra amarilla si hay productos con `CantidadDisponible <= StockMinimo` (propiedades calculadas en `StockDisponibleRow`).

4.3 Layout y feedback UI

`Views/Shared/_Layout.cshtml`:

- Menú de navegación con tag helpers `asp-controller` / `asp-action`.
- Muestra `TempData["Success"]`, `["Warning"]`, `["Error"]`, `["PrecioProductoActualizado"]` (confirmación tras aplicar precio al producto en entrada de inventario).
- CSS global: `site.css`, `dashboard.css` (cargado en todas las páginas).
- Secciones opcionales: `@RenderSection("Styles")`, `@RenderSection("Scripts")`.

4.4 Assets frontend por módulo

Módulo	CSS propio	JS propio	CDN externo
Dashboard	<code>dashboard.css</code>	—	—
Informes	<code>informes.css</code>	—	—
Proveedores	<code>proveedores.css</code>	<code>proveedores.js</code>	—
Inventario Create	—	—	Tom Select 2.3.1
Productos Create/Edit	—	—	Tom Select (según vista)
Ventas Create	—	lógica en vista (JSON carrito)	—

5. Explicación de clases, funciones y servicios principales

5.1 Entidades de dominio (persistidas)

`Categoria` (`Models/Categoria.cs`)

Propiedad	Columna BD	Notas
<code>IdCategoria</code>	<code>idCategoria</code>	PK, identity
<code>NombreCategoria</code>	<code>nombreCategoria</code>	Required
<code>Productos</code>	—	Navegación EF

`Proveedor` (`Models/Proveedor.cs`)

Propiedad	Columna BD	Notas
<code>IdProveedor</code>	<code>idProveedor</code>	PK

Propiedad	Columna BD	Notas
Nombre	nombre	Unique index
Contacto	contacto	Required
MovimientosStock	—	Colección

Producto (Models/Producto.cs)

Propiedad	Columna BD	Notas
IdProducto	idProducto	PK
Nombre, CodigoBarras, Precio, Marca	nombre, etc.	CodigoBarras único
ManejaStock	manejaStock	Si false, venta no descuenta stock
IdCategoria	idCategoria	FK → Categoria

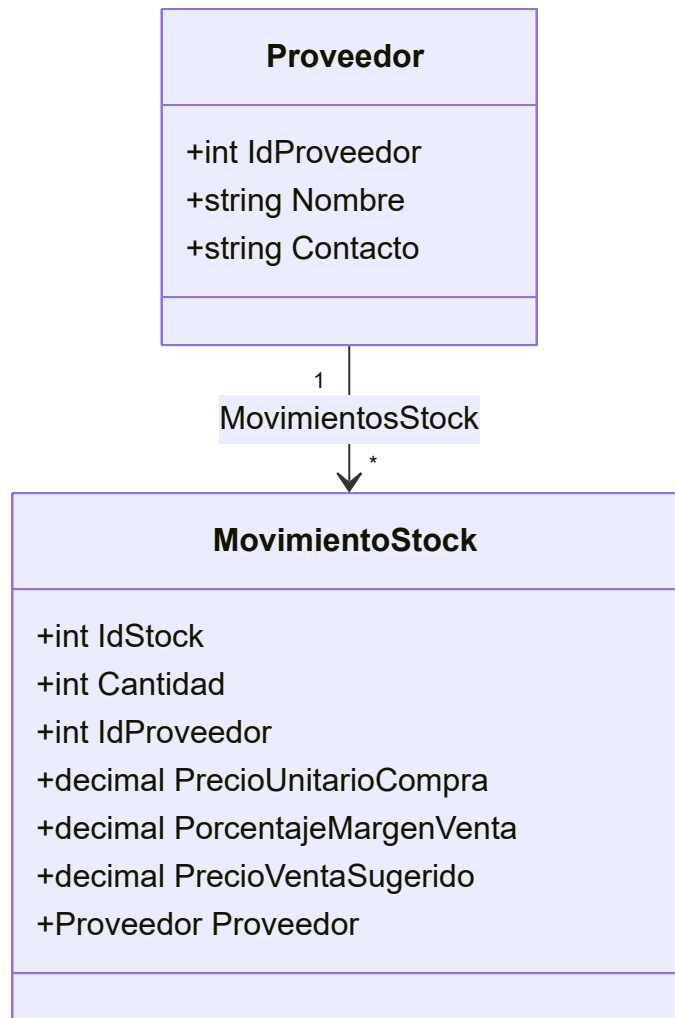
Inventario (Models/Inventario.cs)

Cabecera de movimiento. Fecha es string en formato yyyy-MM-dd (no DateTime en BD).

MovimientoStock (Models/MovimientoStock.cs)

Mapeado a tabla Stock.

Propiedad	Uso
Cantidad	>0 entrada, <0 salida (venta)
StockMinimo	Umbral asociado al movimiento
IdProveedor	Solo entradas (nullable); FK → Proveedor
PrecioUnitarioCompra	Solo entradas (nullable)
PorcentajeMargenVenta	Solo entradas (nullable); margen % de venta
PrecioVentaSugerido	Solo entradas (nullable); compra + (compra × margen / 100), editable en formulario



Venta / DetalleVenta

`Venta.Fecha` como texto `yyyy-MM-dd HH:mm:ss`.

`DetalleVenta` guarda snapshot de `PrecioUnitario` al momento de la venta.

5.2 ViewModels y DTOs (no tablas)

Tipo	Archivo	Propósito
<code>RegistrarEntradaViewModel</code>	<code>Models/RegistrarEntradaViewModel.cs</code>	Formulario alta inventario (proveedor, compra, margen %, precio sugerido editable, <code>AplicarPrecioAlProducto</code>)
<code>EntradaInventarioResultado</code>	<code>Models/EntradaInventarioResultado.cs</code>	Resultado POST entrada (confirmación precio producto)
<code>RegistrarVentaViewModel</code>	<code>Models/RegistrarVentaViewModel.cs</code>	POST venta + lista ítems
<code>InformesViewModel</code> + filas	<code>Models/InformesViewModel.cs</code>	Tablas de informes
<code>InventarioMovimientoRow</code>	<code>Models/InventarioMovimientoRow.cs</code>	Proyección listado inventario (incluye margen y precio venta sugerido)
<code>StockDisponibleRow</code>	<code>Models/StockDisponibleRow.cs</code>	Stock agregado + alertas
<code>ProveedorOperacionResultado</code>	<code>Models/ProveedorOperacionResultado.cs</code>	Respuesta AJAX proveedores
<code>VentaRegistroResultado</code>	<code>Models/VentaRegistroResultado.cs</code>	Id venta + alertas post-venta
<code>ErrorViewModel</code>	<code>Models/ErrorViewModel.cs</code>	Página de error

RegistrarEntradaViewModel — campos del formulario POST

Propiedad	Validación	UI (Inventario/Create)
Fecha	Required	type="date"
IdProducto	Range ≥ 1	Tom Select productos
Cantidad	Required, Range ≥ 1	numérico
StockMinimo	Range ≥ 0	numérico
IdProveedor	Range ≥ 1	Tom Select proveedores (HU6)
PrecioUnitarioCompra	Required, Range > 0.01	input decimal
PorcentajeMargenVenta	Required, Range 0.01–999.99	input + sufijo %; default 30
PrecioVentaSugerido	Required, Range > 0.01	editable; recalculado por JS
AplicarPrecioAlProducto	bool	checkbox

EntradaInventarioResultado — respuesta del servicio

Propiedad	Uso
IdInventario	Cabecera creada
PrecioProductoActualizado	true si se marcó aplicar precio
NuevoPrecioProducto	Valor aplicado a Producto.Precio
NombreProducto	Texto para TempData["PrecioProductoActualizado"]

InventarioMovimientoRow — columnas en Index

Propiedad	Origen
NombreProveedor, PrecioUnitarioCompra	Join Stock → Proveedor
PorcentajeMargenVenta, PrecioVentaSugerido	Columnas en Stock
TotalCompra	Calculado: PrecioUnitarioCompra × Cantidad

5.3 InventarioStockService — API interna

Archivo: Services/InventarioStockService.cs

Constructor: InventarioStockService(AppDbContext db) (primary constructor C# 12).

Método	Firma resumida	Descripción
RegistrarEntradaInventarioAsync	Entrada con proveedor, compra, margen, precio sugerido y flag aplicarPrecioAlProducto	Retorna EntradaInventarioResultado
RegistrarMovimientoAsync	Incluye aplicarPrecioAlProducto	Crea Inventario + Stock; actualiza Producto.Precio solo si el flag es true
CalcularPrecioVentaSugerido	(decimal precioCompra, decimal porcentajeMargen)	compra + (compra × margen / 100), redondeo 2 decimales
ObtenerStockCantidadActualAsync	(int idProducto)	SUM(cantidad) en Stock
GetMovimientosAsync	(int? idCategoria, string? term, int? idProveedor)	Join Inventario/Producto/Categoria/Proveedor → InventarioMovimientoRow
GetStockDisponibleAsync	(int? idCategoria, string? term, int? idProveedor)	Agregación por producto; enriquece último proveedor/precio en memoria
ObtenerStockMinimoActualAsync	(int idProducto)	MAX(stockMinimo)
ObtenerItemAlertaSiStockEnMinimoAsync	(int idProducto)	Para alertas tras venta

Ejemplo de validación en entradas:

```

if (cantidad > 0)
{
    if (!idProveedor.HasValue || idProveedor.Value <= 0)
        throw new InvalidOperationException("Debe seleccionar un proveedor...");
    if (!precioUnitarioCompra.HasValue || precioUnitarioCompra.Value <= 0)
        throw new InvalidOperationException("El precio de compra debe ser mayor a cero.");
    if (!porcentajeMargenVenta.HasValue || porcentajeMargenVenta.Value <= 0)
        throw new InvalidOperationException("El porcentaje de margen de venta debe ser mayor a cero.");
    if (!precioVentaSugerido.HasValue || precioVentaSugerido.Value <= 0)
        throw new InvalidOperationException("El precio de venta sugerido debe ser mayor a cero.");
}

// En el INSERT a Stock se guarda precioVentaSugerido del formulario (no se exige igualdad con la fórmula si el admin ajustó).

if (cantidad > 0 && aplicarPrecioAlProducto && precioVentaSugerido.HasValue)
    producto.Precio = precioVentaSugerido.Value;

```

Fórmula del precio sugerido (cliente y servidor):

```

public static decimal CalcularPrecioVentaSugerido(decimal precioCompra, decimal porcentajeMargen) =>
    Math.Round(precioCompra + precioCompra * porcentajeMargen / 100m, 2, MidpointRounding.AwayFromZero);

```

5.3.1 InventarioController

Archivo: Controllers/InventarioController.cs

Dependencias: ApplicationDbContext (validación FK, SelectList), InventarioStockService.

Acción	Verbo	Descripción
Index	GET	GetMovimientosAsync; filtros categoriaId, proveedorId, q
Create	GET	Prerrequisitos productos/proveedores; RegistrarEntradaViewModel vacío
Create	POST	RegistrarEntradaInventarioAsync; TempData éxito y opcional precio actualizado

POST exitoso — mensajes TempData:

```
TempData["Success"] = "Entrada de inventario registrada correctamente.";
if (resultado.PrecioProductoActualizado)
    TempData["PrecioProductoActualizado"] =
        $"El precio de venta de {nombre} fue actualizado a {precio:C0} (COP).";
```

5.4 VentaService

Archivo: Services/VentaService.cs

Depende de: ApplicationDbContext, InventarioStockService.

Método	Descripción
BuscarProductosVentaAsync	Lista ProductoVentaLookup con stock actual/mínimo
RegistrarVentaAsync	Transacción: inserta Venta, DetalleVenta, movimientos negativos en Stock
ObtenerHistorialVentasAsync	Construye ventaHistorialViewModel

Regla crítica: Si producto.Manejastock == false, no se registra movimiento negativo.

Formato de fechas venta:

```
fecha.ToString("yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture)
```

5.5 ProveedorService (HU6 — prerequisite de inventario)

Método público	Retorno
ListarAsync(string? busqueda)	List<Proveedor>
ObtenerPorIdAsync(int id)	Proveedor?
CrearAsync(Proveedor)	ProveedorOperacionResultado

Método público	Retorno
ActualizarAsync(Proveedor)	ProveedorOperacionResultado

Métodos privados: ValidarCampos, ExisteNombreAsync, ErrorCampo.

5.6 InformesService

Archivo: Services/InformesService.cs

Método principal: ObtenerInformesAsync(desde, hasta, idProductoComparar, idProveedor, idCategoria, q)

Orquesta métodos privados:

- CargarVentasAsync
- CargarMovimientosInventarioAsync
- CargarProductosCatalogoAsync
- CargarCategoriasAsync
- CargarProveedoresAsync
- CargarComparacionPreciosAsync

Nota SQLite: Agregaciones Min/Max sobre decimal en LINQ-to-Entities fallan; el código carga datos con ToListAsync() y agrega en memoria donde aplica (ver comentarios en CargarProductosCatalogoAsync / comparación precios).

InformesSeccion (Models/InformesSeccion.cs)

Constantes para el parámetro seccion en exportación PDF:

Constante	Valor URL	Título en PDF
Ventas	ventas	Ventas — listado
MasVendidos	mas-vendidos	Productos mas vendidos
Salidas	salidas	Salidas de inventario
Entradas	entradas	Entradas de inventario
ComprasProveedor	compras-proveedor	Compras por proveedor
Comparacion	comparacion	Comparacion de precios
Proveedores	proveedores	Resumen de proveedores
Productos	productos	Catalogo de productos
Categorias	categorias	Categorias
Completo	completo	Informe completo (todas las secciones)

```
// Nunca usa "completo" si falta seccion - evita exportar todo por error
public static string Normalizar(string? seccion) { ... }
```

InformesPdfService (Services/InformesPdfService.cs)

Método	Descripción
GenerarPdf(vm, seccion)	Documento QuestPDF A4; cabecera con título de sección
AgregarContenidoPorSeccion	Switch: una tabla/bloque por sección; <code>completo</code> concatena todas

Límite: hasta **80 filas** por tabla larga (nota al pie en PDF si hay más).

InformesController — exportación

```
public async Task<IActionResult> ExportPdf(..., string? seccion)
{
    string seccionNormalizada = InformesSeccion.Normalizar(seccion);
    InformesViewModel vm = await informes.ObtenerInformesAsync(...);
    byte[] pdf = informesPdf.GenerarPdf(vm, seccionNormalizada);
    return File(pdf, "application/pdf", $"Informe_{slug}_{timestamp}.pdf");
}
```

Vista Informes/Index.cshtml — JavaScript

Elemento	Rol
<code>#formExportPdf</code>	GET oculto hacia <code>ExportPdf</code> con filtros + <code>seccion</code>
<code>#btnExportarPdfSeccion</code>	<code>pdfSeccion.value = pestaña activa; form.submit()</code>
<code>#btnExportarPdfCompleto</code>	Fuerza <code>seccion=completo</code>
<code>#btnImprimirSeccion</code>	<code>display:none</code> en paneles inactivos; <code>window.print()</code>
<code>seccionActiva /</code> <code>activarSeccion()</code>	Fuente de verdad; sincroniza los 3 <code></code> de pestañas (Bootstrap no desactiva links entre grupos)

Archivos: `wwwroot/css/informes.css` (@media print, clase `informes-printing-one`).

5.7 Controladores — patrones repetidos

Patrón A: CRUD con ApplicationDbContext (Categorías, Productos)

```
public class CategoriasController(ApplicationDbContext context) : Controller
{
    // Index: AsNoTracking + filtro q + ViewBag.Term
    // Create/Edit GET: return View(entity)
    // Create/Edit POST: ValidateAntiForgeryToken, ModelState, reglas duplicados, TempData,
    RedirectToAction
}
```

```
}
```

Métodos privados de unicidad: `ExisteNombreCategoriaAsync`, `ExisteCodigoBarrasAsync` (Productos).

Patrón B: Controlador + servicio (Proveedores, Inventario, Ventas, Informes)

Injectan servicio por primary constructor y delegan lógica.

Patrón C: AJAX en Proveedores

```
private bool EssolicitudAjax() =>
    string.Equals(Request.Headers["X-Requested-With"], "XMLHttpRequest",
        StringComparison.OrdinalIgnoreCase);
```

Si AJAX → `return Json(resultado)`; si no → vistas + `TempData`.

5.8 Program.cs — funciones estáticas locales

Función	Rol
<code>EnsureSprint3Schema</code>	Parche SQL manual post-migrate (tablas Venta legacy)
<code>EnsureColumnExists</code>	<code>ALTER TABLE</code> si columna falta
<code>ResolveDatabasePath</code>	Ruta canónica de <code>vertice_musicas.db</code>

6. Flujo interno de la aplicación

6.1 Arranque (Program.cs)

```
webApplication.CreateBuilder
    → Register MVC (+ RazorRuntimeCompilation en Development)
    → Register DbContext (SQLite path)
    → Register Services (Scoped)
    → Build
    → Scope: Migrate() + EnsureSprint3Schema()
    → Configure pipeline (ExceptionHandler, HSTS, HTTPS, StaticFiles, Routing, Authorization)
    → MapControllerRoute (default Dashboard/Index)
    → Run
```

6.2 Flujo de stock (concepto central)

```
Stock disponible(producto) = SUM(Stock.cantidad WHERE idProducto = X)
```

- **Entrada inventario:** `cantidad > 0`, con `idProveedor`, `precioUnitarioCompra`, `porcentajeMargenVenta` y `precioVentaSugerido` (calculado en cliente y validado en servidor).
- **Venta:** `cantidad < 0` en nueva cabecera `Inventario` (misma tabla `Stock`).

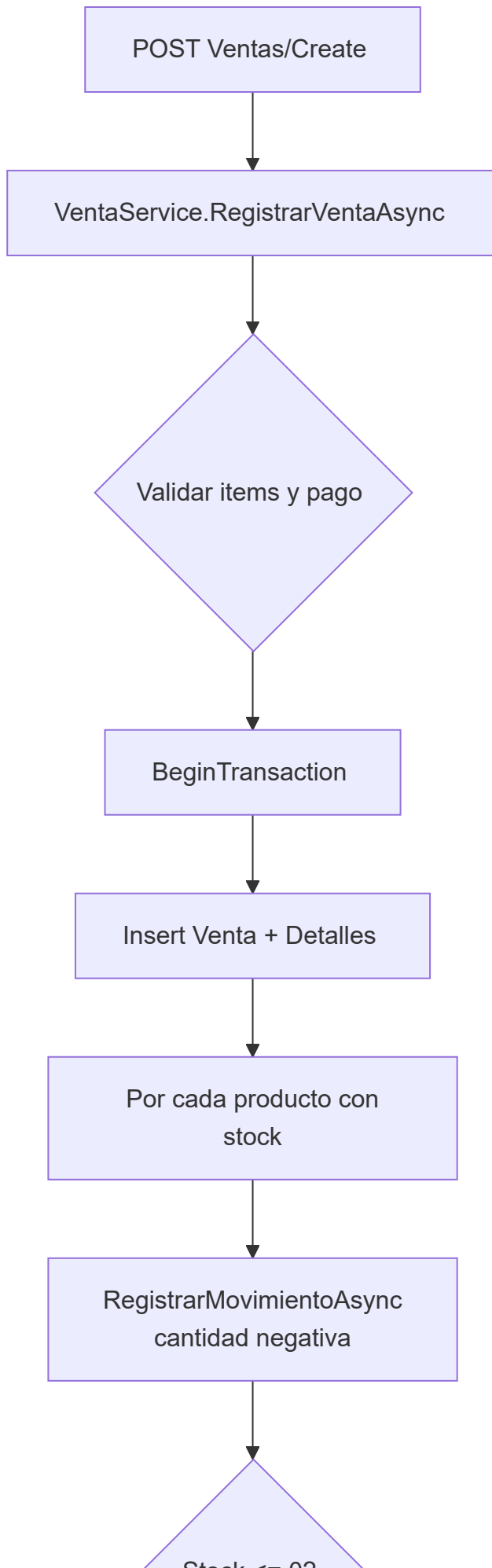
6.3 Flujo POST típico (PRG)

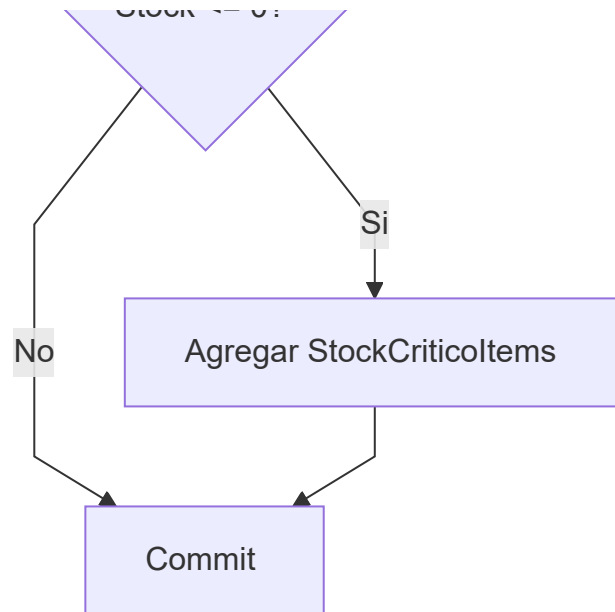
La mayoría de formularios exitosos usan **Post-Redirect-Get**:

```
TempData["Success"] = "...";  
return RedirectToAction(nameof(Index));
```

Excepción: `ventas/Create` redirige a sí mismo tras éxito para mostrar alertas de stock crítico.

6.4 Flujo venta con transacción





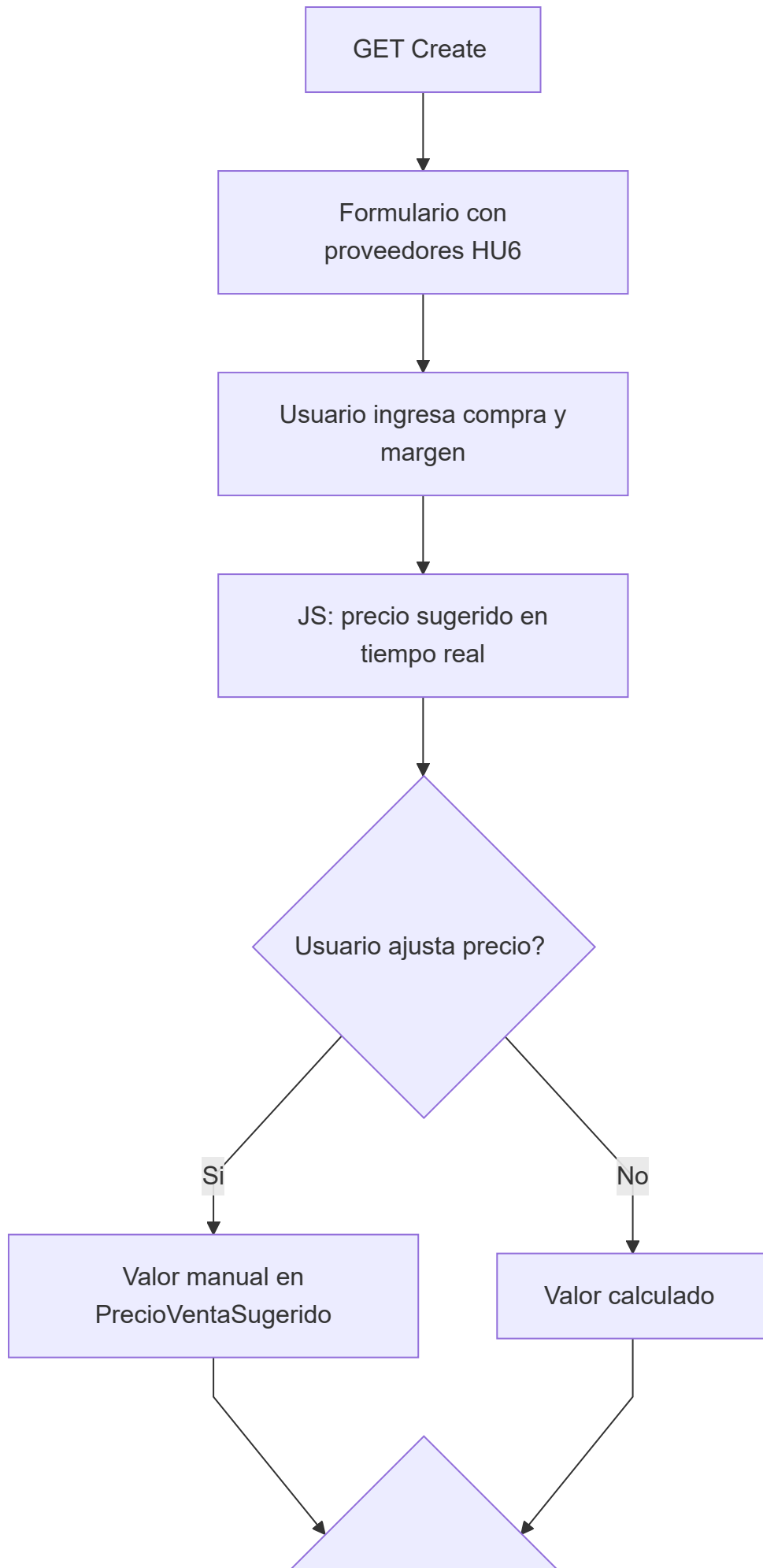
6.5 Flujo entrada de inventario (POST + cálculo UI)

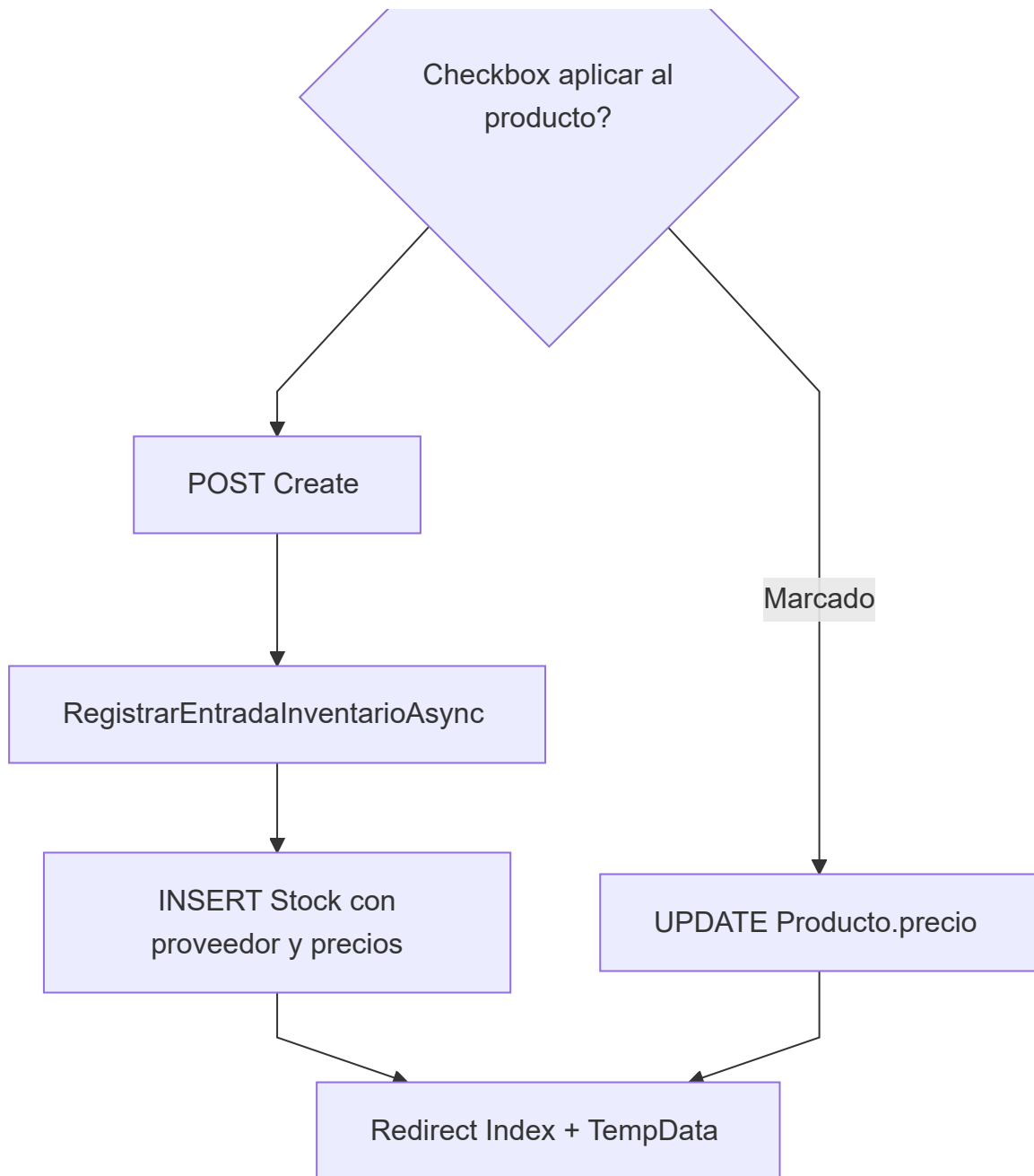
Vista: `views/Inventario/Create.cshtml`

CDN: Tom Select 2.3.1 para producto y proveedor.

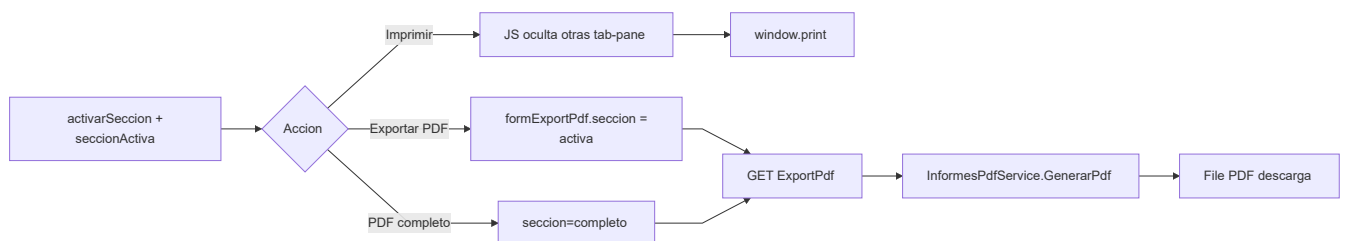
JavaScript (resumen):

Función	Comportamiento
<code>calcularSugerido()</code>	<code>compra + compra * margen / 100</code>
<code>actualizarPrecioVentaSugerido</code>	Actualiza campo si el usuario no editó manualmente
<code>precioventaEditadoManual</code>	Flag al escribir en precio sugerido
<code>btnRestaurarSugerido</code>	Fuerza recálculo desde compra y margen





6.6 Flujo exportación informe por sección



6.7 Flujo proveedor AJAX (proveedores.js)

1. Submit interceptado (`preventDefault`).
2. `fetch` POST con `URLSearchParams` + token antiforgery.
3. Header `X-Requested-With: XMLHttpRequest`.

4. Controller devuelve JSON.
5. JS actualiza tabla DOM sin recargar página.

7. Explicación de rutas y endpoints

7.1 Ruta por defecto

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Dashboard}/{action=Index}/{id?}");
```

URL raíz / → `DashboardController.Index`.

7.2 Tabla completa de acciones

Verbo	Ruta	Controlador	Acción	Notas
GET	<code>/Dashboard</code>	Dashboard	Index	KPIs
GET	<code>/Categorias</code>	Categorias	Index	<code>?q=</code>
GET/POST	<code>/Categorias/Create</code>	Categorias	Create	
GET/POST	<code>/Categorias/Edit/{id}</code>	Categorias	Edit	
GET	<code>/Productos</code>	Productos	Index	<code>?categoriaId=&q=</code>
GET/POST	<code>/Productos/Create</code>	Productos	Create	
GET/POST	<code>/Productos/Edit/{id}</code>	Productos	Edit	
GET	<code>/Proveedores</code>	Proveedores	Index	<code>?q=</code>
GET	<code>/Proveedores/Listar</code>	Proveedores	Listar	JSON
GET	<code>/Proveedores/Obtener/{id}</code>	Proveedores	Obtener	JSON
GET/POST	<code>/Proveedores/Create</code>	Proveedores	Create	POST puede ser JSON
GET/POST	<code>/Proveedores/Edit/{id}</code>	Proveedores	Edit	POST puede ser JSON
GET	<code>/Inventario</code>	Inventario	Index	filtros
GET/POST	<code>/Inventario/Create</code>	Inventario	Create	POST: proveedor, compra, margen, precio sugerido, <code>AplicarPrecioAlProducto</code>
GET	<code>/ConsultaStock</code>	ConsultaStock	Index	filtros
GET	<code>/Ventas/Create</code>	Ventas	Create	carrito JS
GET	<code>/Ventas/Historial</code>	Ventas	Historial	
GET	<code>/Informes</code>	Informes	Index	muchos query params
GET	<code>/Informes/ExportPdf</code>	Informes	ExportPdf	<code>?seccion=productos</code> + filtros; devuelve <code>application/pdf</code>
GET	<code>/Home/Privacy</code>	Home	Privacy	plantilla
GET	<code>/Home/Error</code>	Home	Error	producción

7.3 Query string convenciones

Parámetro	Módulos	Significado
q	Casi todos los índices	Búsqueda texto
categoriaId	Productos, Inventario, ConsultaStock, Informes	Filtro categoría
proveedorId	Inventario, ConsultaStock	Filtro proveedor
desde, hasta	Informes	Rango fechas
idProductoComparar	Informes	Comparación precios un producto
idProveedor, idCategoria	Informes	Filtros informes
seccion	Informes/ExportPdf	Sección del PDF (productos, ventas, completo, etc.)

7.4 Binding de modelos en POST

- Formularios HTML → propiedades del ViewModel/entidad por nombre de campo.
- Ventas: RegistrarVentaviewModel con lista Items (requiere que el JS del cliente envíe índices Items[0].IdProducto, etc.).
- Proveedores AJAX: application/x-www-form-urlencoded, no JSON body puro (pero mismos campos).

8. Manejo de autenticación y autorización

8.1 Estado actual

```
app.UseAuthorization();
```

No hay en el proyecto analizado:

- AddAuthentication()
- [Authorize] en controladores
- ASP.NET Core Identity
- Cookies de sesión de usuario
- Tabla de usuarios

Conclusión: Cualquier cliente con acceso a la URL tiene acceso completo a todas las funciones.

8.2 Medidas relacionadas con seguridad web

Medida	Implementación
CSRF	<code>[ValidateAntiForgeryToken]</code> en POST MVC
HTTPS	<code>UseHttpsRedirection()</code>
HSTS	Solo fuera de Development
Validación entrada	Data Annotations + validación en servicios

8.3 Si debes implementar autenticación (guía orientativa)

No está en el repo; pasos típicos:

1. `dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore`
2. Extender `AppDbContext : IdentityDbContext`
3. `AddIdentity<ApplicationUser, IdentityRole>()` + `AddEntityFrameworkStores`
4. `UseAuthentication()` antes de `UseAuthorization()`
5. `[Authorize]` en controladores o políticas por rol.

Esto requiere diseño (usuarios admin/cajero) no definido en el código actual.

9. Configuración del entorno de desarrollo

9.1 Requisitos

- .NET SDK **8.0.x**
- Editor: Visual Studio 2022, VS Code
- PowerShell 5.1+ (scripts incluidos)
- Navegador moderno

9.2 Perfiles de lanzamiento

Archivo: `Properties/launchSettings.json`

Perfil	URL	Uso recomendado
<code>UnClíc</code>	<code>http://127.0.0.1:5288</code>	Desarrollo diario con puerto fijo
<code>http</code>	puerto dinámico	<code>dotnet run simple</code>
<code>https</code>	HTTPS dinámico	Pruebas SSL local

9.3 Scripts de desarrollo

Inicio con navegador automático:

```
cd VerticeMusicasWeb
.\Iniciar-Vertice-Web.ps1 -Puerto 5288
```

Compilar sin proceso bloqueado:

```
.\build-safe.ps1          # detiene VerticeMuiscaWeb.exe / dotnet y build
.\build-safe.ps1 -Clean # además dotnet clean
```

9.4 Razor Runtime Compilation

En Development, las vistas `.cshtml` se recompilan al guardar sin reiniciar el proceso (paquete `Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation`).

Los cambios en `.cs` de controladores/servicios **sí requieren** recompilar (o hot reload si se usa `dotnet watch` — no configurado en el repo).

9.5 `dotnet watch` (opcional, no preconfigurado)

```
dotnet watch run --launch-profile UnClic
```

No está documentado en scripts del repo; es recomendación para desarrolladores.

10. Dependencias del proyecto

10.1 NuGet (VerticeMusicasWeb.csproj)

```
<PackageReference Include="Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation" Version="8.*" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="8.*" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.*" />
<PackageReference Include="QuestPDF" Version="2026.*" />
```

10.2 Herramientas CLI EF (desarrollo)

Instalación global o local:

```
dotnet tool install --global dotnet-ef
```

Uso desde carpeta del proyecto:

```
dotnet ef migrations add Nombre --context AppDbContext
dotnet ef database update --context AppDbContext
```


10.3 Frontend empaquetado en `wwwroot/lib`

Bootstrap, jQuery, validación unobtrusive — versiones embebidas al crear el proyecto ASP.NET Core, no gestionadas con npm en este repo.

10.4 CDN externos (dependencia de red)

Tom Select en `Views/Inventario/Create.cshtml` (y posiblemente Productos):

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/tom-select@2.3.1/dist/css/tom-select.bootstrap5.min.css" />
```

Sin conexión a internet, esos selectores avanzados pueden fallar.

11. Convenciones de código utilizadas

11.1 C# / .NET

Convención	Ejemplo en proyecto
Primary constructors	<code>public class X(AppDbContext db) : Controller</code>
<code>async / await</code> en I/O	Controladores y servicios
Nullable reference types	<code>nullable</code> en migraciones; propiedades <code>string?</code>
Namespace	<code>VerticeMusicasWeb.{Controllers,Models,Services,Data}</code>
Nombres en español	<code>NombreCategoria</code> , <code>IdProveedor</code>
Métodos privados validación	<code>ExisteNombreCategoriaAsync</code> , <code>ExisteNombreAsync</code>

11.2 Base de datos (EF Fluent API)

Convención	Ejemplo
Tablas en español singular	<code>Producto</code> , <code>Categoria</code> , <code>Stock</code>
Columnas camelCase español	<code>idProducto</code> , <code>nombreCategoria</code>
Propiedades C# PascalCase	<code>IdProducto</code> , <code>NombreCategoria</code>
FK restrict en catálogo	<code>Producto</code> → <code>Categoria</code>
Cascade en detalles venta	<code>DetalleVenta</code> → <code>Venta</code>

11.3 Vistas y UI

Convención	Uso
<code>TempData["Success"]</code> / <code>["Error"]</code> / <code>["PrecioProductoActualizado"]</code>	Feedback tras redirect
<code>ViewBag</code>	Filtros, listas SelectList, JSON embebido
Clases CSS propias	<code>card-modern</code> , <code>btn-modern-primary</code> , <code>btn-modern-secondary</code>
Filtros GET	<code>method="get"</code> en formularios de búsqueda
Bootstrap 5	Grid, tabs (Informes), alerts

11.4 Parámetros de filtro estándar

- `q` — texto libre.
- `categoriaId`, `proveedorId` — int opcional.
- `desde`, `hasta` — `DateTime?` en informes.

12. Patrones de diseño identificados

Patrón	Dónde	Evidencia
MVC	Toda la app	Controllers + Views
Repository-like (ligero)	Servicios con <code>AppDbContext</code>	No hay interfaz <code>IRepository</code>
Service Layer	<code>Services/*.cs</code>	Lógica fuera de controladores
DTO / ViewModel	<code>*Row</code> , <code>*ViewModel</code> , <code>*Resultado</code>	No exponer entidades a todas las vistas
Primary constructor DI	Controladores y servicios	C# 12 style
Post-Redirect-Get	POST exitosos	<code>RedirectToAction</code> + <code>TempData</code>
ViewComponent	AlertaStock	Reutilización en layout
Unit of Work (parcial)	<code>IDbContextTransaction</code> en <code>VentaService</code> e <code>InventarioStockService</code>	Transacciones explícitas

Patrón	Dónde	Evidencia
Strategy (implícita)	<code>EsSolicitudAjax()</code>	JSON vs HTML en Proveedores

No identificado: CQRS, Mediator, Clean Architecture con proyectos separados, Event Sourcing.

13. Conexión con base de datos

13.1 Configuración real vs appsettings

En código (`Program.cs`):

```
string dbPath = ResolveDatabasePath(builder.Environment.ContentRootPath);
options.UseSqlite($"Data Source={dbPath}");
```

En `appsettings.json`:

```
"ConnectionStrings": {
  "DefaultConnection": "Data Source=vertice_musicas.db"
}
```

La cadena `DefaultConnection` **no se consume** en `Program.cs` actualmente. No confundir al cambiar configuración.

13.2 AppDbContext

```
public class AppDbContext(DbContextOptions<AppDbContext> options) : DbContext(options)
{
    public DbSet<Categoria> Categorias => Set<Categoria>();
    public DbSet<Proveedor> Proveedores => Set<Proveedor>();
    // ...
}
```

Configuración completa en `OnModelCreating` — no hay atributos `[Table]` en entidades, solo Fluent API.

13.3 Consultas típicas

Escenario	Patrón usado
Listados	<code>.AsNoTracking()</code>
Edición con tracking	<code>.FindAsync()</code> o <code>.FirstOrDefaultAsync()</code> sin <code>AsNoTracking</code>
Búsqueda	<code>EF.Functions.Like</code> con <code>%term%</code>
Agregaciones stock	<code>.SumAsync</code> , <code>.MaxAsync</code> en LINQ
Includes	<code>.Include().ThenInclude()</code> en ventas/historial

13.4 Migraciones y parche legacy

Al arrancar:

1. `Database.Migrate()` — aplica migraciones EF pendientes.
2. `EnsureSprint3Schema(db)` — SQL crudo para compatibilidad.

Si agregas columnas nuevas, **prefiere migraciones EF** y evita duplicar en `EnsureSprint3Schema` salvo compatibilidad legacy.

Migraciones relevantes para entrada de inventario:

Migración	Cambio
<code>20260525022731_AgregarTablaProveedor</code>	Tabla <code>Proveedor</code>
<code>20260525024254_ProveedorEnMovimientoStock</code>	<code>Stock.idProveedor</code> , <code>precioUnitarioCompra</code> , FK → <code>Proveedor</code>
<code>20260525175230_MargenYPrecioVentaEnMovimientoStock</code>	<code>porcentajeMargenVenta</code> , <code>precioVentaSugerido</code> en <code>Stock</code>

Mapeo EF en `AppDbContext.OnModelCreating` → entidad `MovimientoStock`, tabla `Stock`.

13.5 Tablas legacy no mapeadas

Migraciones crearon `ConfiguracionAlertaStock` y columnas relacionadas. **No están en** `AppDbContext`. Pueden existir en BD pero la aplicación actual no las usa.

14. Manejo de errores y logs

14.1 Capas de error

Capa	Mecanismo	Ejemplo
Validación modelo	<code>ModelState</code> , Data Annotations	Campo obligatorio producto
Reglas negocio	Excepciones <code>InvalidOperationException</code>	Venta sin ítems
Servicios CRUD	<code>ProveedorOperacionResultado.Exito == false</code>	AJAX
Persistencia	<code>try/catch</code> genérico → <code>TempData</code>	"No se pudo guardar"
Pipeline ASP.NET	<code>UseExceptionHandler("/Home/Error")</code>	Producción

14.2 Mensajes al usuario

Layout muestra alertas Bootstrap desde `TempData`:

Clave	Estilo	Cuándo
Success	alert-success	Operación OK genérica
Error	alert-danger	Fallo validación o excepción
Warning	alert-warning	Avisos
PrecioProductoActualizado	alert-info	Tras entrada inventario con AplicarPrecioAlProducto

Proveedores AJAX: `#proveedor-alert` + spans `[data-error-for]`.

Ventas: `ModelState.AddModelError(string.Empty, ex.Message)` muestra excepciones de servicio.

14.3 Logging

`appsettings.json`:

```
"LogLevel": {
  "Default": "Information",
  "Microsoft.AspNetCore": "Warning"
}
```

Salida típica en consola: comandos SQL de EF cuando nivel Information (comportamiento por defecto de EF).

No hay Serilog, archivos de log ni Application Insights configurados en el repo.

14.4 Página de error

`HomeController.Error` → `ErrorViewModel` con `RequestId` / trace identifier.

15. Cómo agregar nuevas funcionalidades

15.1 Árbol de decisión

```
¿Nueva tabla en BD?
  → Modelo + AppDbContext + Migración EF
¿Lógica compleja o transacciones?
  → Service en Services/ + registro en Program.cs
¿Solo CRUD simple?
  → Controlador directo (como Categorías)
¿Nueva pantalla?
  → Views/{Modulo}/ + enlace en _Layout.cshtml
¿Filtros en listado?
  → Parámetros GET + ViewBag SelectList
```

15.2 Checklist de implementación

1. [] Modelo y mapeo en `OnModelCreating`
2. [] `dotnet ef migrations add ...`
3. [] Servicio (si aplica) con pruebas manuales de reglas
4. [] Controlador con POST `[ValidateAntiForgeryToken]`
5. [] Vistas con `_ValidationScriptsPartial` en Scripts
6. [] Enlace en menú `_Layout.cshtml`
7. [] CSS/JS en `wwwroot` si el módulo lo requiere
8. [] Actualizar Informes/Dashboard si el dato debe aparecer en KPIs

15.3 Ejemplo: agregar campo `Email` a Proveedor

1. Propiedad en `Models/Proveedor.cs` + anotación `[EmailAddress]`.
2. Mapeo columna en `AppDbContext`.
3. `dotnet ef migrations add ProveedorEmail`.
4. Actualizar `ProveedorService.ValidarCampos`.
5. Actualizar vistas `Index`, `Create`, `Edit`, `proveedores.js` (filas AJAX).
6. Probar Create/Edit AJAX y no-AJAX.

15.4 Integrar con inventario u otros módulos

Si la funcionalidad toca stock, **siempre** pasar por `InventarioStockService` para no romper la regla de signo de cantidad ni transacciones.

16. Cómo crear nuevos endpoints o módulos

16.1 Nuevo módulo MVC completo (plantilla)

1. **Modelo** — `Models/Ciente.cs` (ejemplo).

2. **DbContext:**

```
public DbSet<Ciente> Cientes => Set<Ciente>();  
// + bloque OnModelCreating
```

3. **Migración:**

```
dotnet ef migrations add AgregarCiente --context AppDbContext
```

4. **Servicio (recomendado):** `Services/CienteService.cs`

5. **Registro DI:**

```
builder.Services.AddScoped<ClienteService>();
```

6. Controlador: `Controllers/CientesController.cs`

```
namespace VerticeMusicasWeb.Controllers;

public class CientesController(ClienteService service) : Controller
{
    public async Task<IActionResult> Index() { ... }
}
```

7. Vistas: `Views/Cientes/Index.cshtml`, `Create.cshtml`, `Edit.cshtml`

8. Menú en `_Layout.cshtml`:

```
<a class="menu-link" asp-controller="Cientes" asp-action="Index">Cientes</a>
```

16.2 Nuevo endpoint JSON (estilo Proveedores)

```
[HttpGet]
public async Task<IActionResult> Listar(string? q)
{
    var datos = await _service.ListarAsync(q);
    return Json(datos);
}

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(Cliente cliente)
{
    var resultado = await _service.CrearAsync(cliente);
    if (EsSolicitudAjax())
        return Json(resultado);
    // ... vista
}
```

Cliente JS:

```
headers: {
    'X-Requested-With': 'XMLHttpRequest',
    'Content-Type': 'application/x-www-form-urlencoded'
}
```

16.3 Nueva acción en controlador existente

Ejemplo: exportar CSV desde Informes.

1. Añadir método `ExportarCsv` en `InformesController`.
2. Reutilizar `InformesService.ObtenerInformesAsync`.

3. Devolver `File(bytes, "text/csv", "informes.csv")` en lugar de `view()`.

No requiere nueva vista si solo descarga archivo.

16.4 API REST formal (no existe hoy)

Requeriría nuevo proyecto o:

```
builder.Services.AddControllers(); // API
// + MapControllers() separado de MVC
```

No está implementado; sería cambio arquitectónico mayor.

17. Recomendaciones para mantenimiento

1. **Detener proceso antes de build** en Windows (`build-safe.ps1`).
2. **Una sola BD canónica:** `vertice_musicas.db` en raíz del proyecto web.
3. **No editar** `AppDbContextModelSnapshot` **a mano** — usar herramientas EF.
4. **Tras cambiar modelo:** `dotnet ef migrations add` + probar en BD vacía y BD con datos.
5. **Mantener paridad** entre validación en servicio y Data Annotations en ViewModels.
6. **Documentar reglas de negocio** nuevas en comentarios solo si no son obvias (el código prefiere ser autoexplicativo).
7. **Revisar impacto en Informes** cuando se toque `Stock` o `Venta`.
8. **Evitar agregar lógica de negocio en vistas** — ya hay precedente con JSON en Ventas; para módulos nuevos preferir servicios + JS pequeño.

18. Buenas prácticas del proyecto

18.1 Hacer (alineado al código existente)

- Usar `AsNoTracking()` en listados de solo lectura.
- Usar `Trim()` en nombres antes de guardar.
- Validar duplicados con métodos privados `Existe*Async` en controladores o servicios.
- Usar `TempData` para mensajes tras redirect exitoso.
- Usar transacciones en operaciones multi-tabla (ventas, inventario).
- Centralizar stock en `InventarioStockService`.
- Filtrar con `EF.Functions.Like` para búsquedas SQLite.

18.2 Evitar (rompe convenciones o runtime)

- Agregar API sin `[ValidateAntiForgeryToken]` en formularios MVC.
- Escribir SQL manual fuera de migraciones sin motivo legacy.
- Usar `DateTime` en BD sin acordar formato string existente para `Inventario.Fecha` y `Venta.Fecha`.
- Agregar `Min/Max` de `decimal` en consultas EF traducidas a SQLite (ver `InformesService`).
- Usar colección `[]` en archivos `.cshtml` si el compilador Razor falla (preferir `new List<T>()` en bloques `@{}` de vistas).
- Confiar en `appsettings` `ConnectionStrings` sin cablear en `Program.cs`.

18.3 Estilo de commits sugerido

```
feat(inventario): agregar filtro por rango de fechas
fix(ventas): validar stock antes de confirmar carrito
docs: actualizar manual programador
```

No hay convención documentada en repo; sugerencia para equipos.

19. Problemas comunes y soluciones

19.1 `dotnet build` falla: archivo en uso

Síntoma:

```
error MSB3027: No se pudo copiar ... VerticeMiscaWeb.exe because it is being used by
another process
```

Causa: La app sigue ejecutándose.

Solución:

```
.\build-safe.ps1
# o manualmente:
taskkill /IM VerticeMiscaWeb.exe /F
```

19.2 La UI no refleja cambios en vistas

Causa: Proceso viejo ejecutándose o sin Runtime Compilation.

Solución:

1. Detener y `dotnet build` + `dotnet run`.
 2. Verificar `ASPNETCORE_ENVIRONMENT=Development`.
 3. Confirmar paquete `Razor.RuntimeCompilation` instalado.
-

19.3 HTTP 500 en Dashboard u otra vista tras editar `.cshtml`

Síntoma: `CompilationFailedException` en Razor.

Ejemplo real corregido: operador `?? []` en `@{}` — no soportado en todas las versiones de compilador Razor.

Solución:

```
// Evitar en .cshtml:
var x = ViewBag.Algo as List<T> ?? [];

// Preferir:
var x = ViewBag.Algo as List<T> ?? new List<T>();
```

19.4 Error EF SQLite: `Min` on decimal

Síntoma:

```
SQLite cannot apply aggregate operator 'Min' on expressions of type 'decimal'
```

Solución: Materializar antes de agregar:

```
var datos = await query.ToListAsync();
var min = datos.Min(x => x.Precio);
```

Patrón ya usado en `InformesService.CargarProductosCatalogoAsync`.

19.5 Migración no se aplica

Verificar:

1. `__EFMigrationsHistory` en SQLite.
2. Logs al arranque (`Applying migration...`).
3. Permisos escritura en carpeta de `vertice_musicas.db`.

Forzar desarrollo:

```
dotnet ef database update --context AppDbContext
```

19.6 Entrada inventario rechazada o precio no aplicado

Mensajes típicos del servicio:

Excepción / error	Causa
Debe seleccionar un proveedor	<code>IdProveedor</code> vacío o inválido
El precio de compra debe ser mayor a cero	<code>PrecioUnitarioCompra</code> ≤ 0
El porcentaje de margen... debe ser mayor a cero	<code>PorcentajeMargenVenta</code> ≤ 0
El precio de venta sugerido debe ser mayor a cero	<code>PrecioVentaSugerido</code> ≤ 0

El precio del producto no cambió tras la entrada: solo se actualiza `Producto.Precio` si el POST incluye `AplicarPrecioAlProducto=true` (checkbox marcado). Sin el checkbox, el movimiento guarda precios en `Stock` pero el catálogo no cambia.

Redirección a Proveedores al abrir Create: no hay proveedores en BD; crear al menos uno en `/Proveedores`.

Archivos a revisar: `RegistrarEntradaViewModel.cs`, `Inventario/Create.cshtml`, `InventarioStockService.RegistrarEntradaInventarioAsync`.

19.7 Stock negativo tras venta

Comportamiento esperado si se vende más de lo disponible: movimiento negativo y posible alerta en `TempData` / `StockCriticoItems`.

Revisar: `Producto.ManejaStock`, suma histórica de movimientos.

19.8 PDF de informes exporta todo en lugar de una sección

Síntoma: al pulsar «Exportar PDF» en la pestaña Productos, el PDF incluye ventas, entradas, etc.

Causas corregidas en código:

1. `InformesSeccion.EsValida` inicializaba `normalizada = completo` (eliminado; usar `Normalizar()`).
2. Enlace PDF con `href="#"` sin actualizar `seccion` (reemplazado por formulario GET `#formExportPdf`).
3. Tres grupos de pestañas Bootstrap: varios `.nav-link.active` y varios `.tab-pane.active` a la vez (impresión mostraba todo). Solución: `activarSeccion()` + variable `seccionActiva`.

Verificar: nombre del archivo `Informe_productos_...pdf` y título en cabecera del PDF («Catalogo de productos»). Si sigue saliendo todo, reiniciar la aplicación para cargar el ensamblado nuevo.

Solo informe integral: botón explícito «PDF completo» (`seccion=completo`).

19.9 Tom Select no funciona offline

Causa: CDN en `Inventario/Create.cshtml`.

Solución: Descargar Tom Select a `wwwroot/lib` y referenciar localmente.

19.10 Nombres de proceso / ensamblado confusos

Buscar: `VerticeMuiscaWeb`, `VerticeMusicasWeb`, `VerticeMusicaWeb` (typo en scripts `build-safe.ps1`).

20. Conclusiones

Vertice Muisca es un **monolito ASP.NET Core MVC** con separación pragmática de lógica en **cuatro servicios** y CRUD directo en catálogos simples. El dominio gira en torno a **movimientos de stock firmados y ventas transaccionales**.

Para el programador, los puntos críticos del código son:

1. `InventarioStockService` — única fuente de verdad para altas/bajas de stock.
2. `VentaService` — transacciones de venta y coordinación con inventario.
3. `AppDbContext` + **migraciones** — esquema y convenciones de columnas en español.
4. `Program.cs` — ruta de BD y parches de esquema legacy.

El sistema es extensible siguiendo los patrones de módulos existentes (controlador → servicio opcional → vista → menú). Las mayores carencias técnicas para entornos productivos son **autenticación, configuración centralizada de conexión y separación formal de API** si se requiere integración externa.

Este manual debe actualizarse cuando se agreguen módulos, entidades o cambios en el pipeline de arranque.

Apéndice A — Inventario de archivos de código de aplicación

Carpeta	Cantidad aprox.	Rol
<code>Controllers/</code>	9 archivos	Entrada HTTP
<code>Services/</code>	5 archivos	Lógica aplicación
<code>Models/</code>	~21 archivos	Entidades y VM
<code>Views/</code>	~20+ <code>.cshtml</code>	UI
<code>Migrations/</code>	9+ migraciones + snapshot	Esquema BD (incl. <code>MargenYPrecioVentaEnMovimientoStock</code>)
<code>Data/</code>	1 archivo	EF Core
<code>ViewComponents/</code>	1 archivo	UI compartida

Apéndice B — Referencia rápida de archivos clave

Archivo	Rol
<code>Program.cs</code>	Arranque, DI, migrate, pipeline

Archivo	Rol
<code>Data/AppDbContext.cs</code>	Mapeo relacional
<code>Services/InventarioStockService.cs</code>	Stock e inventario
<code>Services/VentaService.cs</code>	Ventas
<code>Services/ProveedorService.cs</code>	Proveedores
<code>Services/InformesService.cs</code>	Datos de reportes
<code>Services/InformesPdfService.cs</code>	PDF por sección (QuestPDF)
<code>Models/InformesSeccion.cs</code>	Constantes <code>seccion</code> para exportación
<code>Views/Shared/_Layout.cshtml</code>	Menú global
<code>wwwroot/css/site.css</code>	Estilos base
<code>Properties/launchSettings.json</code>	Perfiles de ejecución
<code>build-safe.ps1</code>	Build en Windows sin lock

Manual de programador basado en análisis estático del repositorio VerticeMusicasWeb. Versión alineada al código disponible en mayo 2026.